

Carmina: Introducing Programming to Latin Poetry

Suh Young Choi



Coding the algorithms that make analysis of Latin poetry possible.

Computational thinking is a problem-solving framework traditionally associated with computer science, but it can be applied to other fields such as humanities computing. Carmina, whose name comes from the Latin word for “songs” or “poems”, is a new library under development to demonstrate the algorithmic nature of analytic tasks related to Latin poetry. Carmina has practical use in automating processes like scansion and alliteration finding; further, it has educational utility for classical scholars by translating familiar tasks to a coding context.

Design Philosophy

Carmina is intended for both practical and pedagogical use. To keep the code readable and comprehensible for new programmers, we kept the following design principles:

- **Simplicity:** Most of Carmina is written in base Python without using other libraries or dependencies.
- **Modularity:** The current version of Carmina has separate modules for different tasks (file parsing, scansion, and alliteration finding)
- **Comprehensibility:** Traditional manual methods of doing scansion and alliteration finding were translated to code rather than coming up with new algorithms.

Scansion

Latin poetry follows specific *meters*, or rhythmic patterns of long and short syllables. *Scansion* is the process of determining where the long and short syllables fall within lines of poetry.

Unlike meters in English poetry, meters of Latin poetry may not have a fixed number of words or syllables per line.

Ārnā vīr|ūmq̄e cān|ō, || Trō|iaē quī| p̄mūs āb| ōrīs
Ītālī|ām fāt|ō || prōfūg|ūs Lāv|īniāquē| vēnīt
lītōrā|, mūltum īl|le ēt || tēr|rīs iāc|tātūs ēt| āltō
vī sūp̄r|ūm, saēv|aē || mēmōr|ēm Iūn|ōnis ōb| īrām,

Fig. 1: The first four lines of Virgil's Aeneid, with scansion. – indicates a long syllable, while ~ indicates a short syllable. A single | indicates a break between feet, while the double || indicates the caesura. Credit: Hands Up Education AP Vergil Resources

Scanning poetry by hand follows these steps:

- Determine the syllables of each line.
- Determine whether the syllable is long or short. Syllables can be long/short by nature of their vowels or by position in the text
- Determine elisions (omissions of certain syllables in the meter).
- Verify that the resulting feet are appropriate for the meter.

```
for line in lines:
    syllables = line.syllabify()
    for syllable in syllables:
        syllable.mark_long() if is_long(syllable) \
            or syllable.elide(next_syllable) \
            else syllable.mark_short()
    line.verify_meter()
```

Fig. 2: Pseudocode for scanning a line of poetry, following the steps outlined above.

Alliteration

Alliteration is the repetition of the same letter at the beginning of words (initial) or syllables (medial).

In Latin poetry, alliteration typically refers to three or more words or syllables in close proximity that share a common initial letter. Alliteration can also be mixed between two letters (such as recurring R and G).

ō Tite, Tūte, Tatī, Tibi Tanta, Tyranne, Tulistī!

Fig. 3: A famous quote from Ennius, featuring alliteration (both initial and medial) on the letter T. The repeated letter is marked in bold. The line translates to, “O Titus Tatius, you yourself have brought such great (misfortunes) upon yourself, king!”

Determining alliteration by hand follows these steps:

- Determine the syllables of each line.
- Determine initial letters for each syllable.
- If the same letter is repeated in syllables appearing “close enough” to each other, they are marked as being within the same alliteration.

```
all_alliteration = {}
for line in lines:
    alliterations = {}
    syllables = line.syllabify()
    for syllable in syllables:
        initial_letter = syllable[0]
        if initial_letter not in alliterations:
            alliterations[initial_letter] = [syllable]
        else:
            alliterations[initial_letter].append(syllable)
    all_alliteration[line_number] = alliterations
```

Fig. 4: Pseudocode for finding alliterations within a line, following the steps outlined above. Dictionaries are used to map initial/medial letters to their syllables.

File Parsing

Carmina uses Python’s built-in file reading with additional processes for cleaning files before use. We implemented the file parsing functions without using external libraries to keep the process transparent for the users.

Future Directions

The initial version of Carmina is still under development and currently supports parsing for plaintext and XML files; scansion for dactylic hexameter; and initial-letter alliteration finding. We hope that future versions of Carmina will support the following:

- Scansion for other kinds of meters, including (but not limited to) hendecasyllabic, elegiac couplet, and Saturnian verse.
- Text matching and reuse, both for lemmas and exact matches.
- Expanding the alliteration finder to determine alliteration within words in addition to between words.
- Detection of other poetic devices including (but not limited to) chiasmus, synchysis, assonance, and anaphora

Acknowledgments

Many thanks to Hui-Hsuan Chan, Elizabeth Nova, and Simon Dovan Nguyen as collaborators in the original Carmina development team; to David Beck, Bryna Hazelton, Nels Schimek, and Natalie Robbins as advisors during original development.